GREENTRACE 2.0

CORE IMPLEMENTATION PROMPT

Core files, roles, inputs, contract logic and end-to-end flow

Purpose: Use this focused prompt when building or repairing core files. Use with the Master Prompt for full context.

CORE PROMPT — paste into your coding agent

- Act as a senior full-stack developer and Tezos SmartPy engineer. Inspect the workspace first, preserve working code, verify current package APIs and testnet support, and implement one stage at a time. Never fabricate addresses, transaction hashes, confirmations, test results or successful network interactions.
- Build GreenTrace 2.0. Students submit campus sustainability activities from their own Tezos wallets. Authorized organizers review and approve/reject using their own wallets. Exactly two authenticated roles: STUDENT and ORGANIZER. Public QR verification is read-only. Use SmartPy, Taquito and a compatible Beacon wallet integration. Do not replace wallet submission with college login or a backend relayer.

1. Core files and responsibilities

- **contracts/greentrace.py:** SmartPy data model, administrator/organizer permissions, submit/approve/reject entrypoints, duplicate policy and status transitions.
- **contracts/tests/:** contract tests for valid operations, access control, duplicate IDs, self-approval, invalid input and repeated decisions.
- **frontend/src/hooks/useWallet.ts:** connect/disconnect, current address, account changes, network state and wallet errors.
- **frontend/src/services/tezos.ts:** wallet/provider setup, contract access, entrypoint calls, transaction hashes, confirmation state and RPC errors.
- **frontend/src/utils/hash.ts:** SHA-256 from actual evidence bytes, lowercase hex encoding.
- **frontend/src/services/api.ts:** typed FastAPI calls and consistent error handling.
- **frontend/src/types/activity.ts:** shared category, quantity, unit, status, address, hash and timestamp types.
- **frontend/src/pages/SubmitActivity.tsx:** student form, validation, upload, duplicate warning, wallet signature and result.
- **frontend/src/pages/StudentDashboard.tsx:** student activity history and counts.
- **frontend/src/pages/OrganizerDashboard.tsx:** pending queue, evidence review, duplicate warning, approve/reject.
- **frontend/src/pages/VerifyActivity.tsx:** public read-only verification against configured contract/network.
- **backend/app/main.py:** FastAPI app setup and routes.
- **backend/app/database.py and models.py:** SQLite setup and persistence models.
- **backend/app/routes/evidence.py:** validated upload and duplicate checks.
- **backend/app/routes/activities.py:** activity metadata and history.
- **backend/app/routes/public.py:** public-safe record lookup without private evidence or personal data.
- **backend/app/services/hashing.py:** server-side SHA-256 calculation and verification.
- **README.md:** reproducible setup, deployment, testing and troubleshooting.

2. Student inputs and generated values

- **Inputs:** wallet address from the connected provider; category dropdown (Recycling, Tree planting, Energy conservation, Water conservation, Waste collection, Other); positive quantity; compatible unit; short description; evidence image; optional reported activity date.
- **Generated:** unique activity ID, SHA-256 hash, contract timestamp, SUBMITTED status, actual transaction hash.
- **Validation:** category/unit compatibility; positive quantity; file size/type/content; description; wallet connected; correct network; duplicate hash and ID. Disable repeat submits while upload or transaction is in progress.
- **Success:** student signs the real transaction, waits for confirmation, then sees actual ID, status and transaction reference.

3. Organizer inputs and permissions

- **Inputs:** authorized connected wallet, selected pending activity, Approve or Reject, optional rejection reason stored off-chain.
- **Checks before action:** contract authorizes wallet; activity exists and remains SUBMITTED; organizer address differs from submitter.
- **Success:** organizer signs the operation; contract changes status; app waits for confirmation and displays actual transaction reference.

- Frontend role checks are only for usability. Contract enforces permissions, self-approval prevention and valid state transitions.

4. Smart contract data model

- Each record contains activity_id, evidence_hash, category, quantity, unit, status, submitter_address, approver_address, submitted_at, approved_at, and optional superseding-record reference.
- Use a suitable SmartPy map/big-map keyed by activity ID. No photos, names, emails or private reviewer notes on-chain.
- submit_activity derives submitter from Tezos sender, validates unique ID/input/eligibility/duplicate policy, sets SUBMITTED and records contract timestamp.
- approve_activity requires authorized organizer, existing SUBMITTED record and sender different from submitter; derives approver from sender, records timestamp and sets APPROVED.
- reject_activity requires authorized organizer and SUBMITTED record; derives approver from sender and sets REJECTED without overwriting a previous decision.
- add_organizer/remove_organizer are administrator-only. Test every permission boundary. Store detailed rejection text off-chain.

5. Backend and database essentials

- Use FastAPI, Pydantic and SQLite. Suggested endpoints: GET /health; POST /api/evidence; GET /api/evidence/{hash}/duplicate-check; GET /api/activities; GET /api/activities/{id}; GET /api/activities/{id}/history; GET /api/public/activities/{id}. Document any changes.
- Validate uploads, calculate server-side hash, detect duplicates, protect stored files, maintain metadata, save transaction references, serve public-safe records and reconcile chain/cache.
- Suggested activities fields: activity_id, submitter_address, category, quantity, unit, description, evidence_hash, evidence_path, status_cache, submission_tx_hash, approval_tx_hash, submitted_at, approved_at, rejection_reason. Add indexes and unique constraints.
- Make retries safe if transaction confirms but database update fails. Database edits must never substitute for contract operations. Backend must not sign or relay student/organizer operations.

6. Required pages

- Landing page and Connect Wallet; wallet status component; student dashboard; validated submission form; activity details; organizer dashboard with pending queue and decisions; public QR verification page.
- Public verification shows public fields, status, hash, network, contract address and explorer links. It never exposes private photographs, college email or unnecessary personal information.

7. Environment and security

- Create .env.example placeholders for API URL, verified network, RPC URL, deployed contract address, explorer URL, database path, upload directory, size limit and CORS origins.
- VITE_ variables are public. Never put private keys, recovery phrases or signing secrets in frontend variables, Git, logs or database.
- Verify current official testnet, RPC, faucet, wallet compatibility, SmartPy and Taquito APIs. Do not assume Ghostnet is supported or invent deployment values.
- Handle wallet rejection, unsupported network, insufficient funds, RPC failure, pending transactions, confirmation timeout, missing contract, invalid ID and QR verification failure.

8. Build order

- 1. Inspect/scaffold. 2. Frontend routes/forms. 3. Wallet connection. 4. SmartPy contract/tests. 5. Verify testnet/deployment. 6. Student wallet-signed submission. 7. Upload, SHA-256 and duplicates. 8. Organizer wallet-signed decisions. 9. SQLite and reconciliation. 10. Dashboard/history. 11. QR verification. 12. Security review, end-to-end tests and documentation.

9. Acceptance tests

- Student connects wallet and submits valid activity with a real testnet transaction.

- Exact duplicate file is detected; do not claim modified images always match.
- Authorized organizer signs approval/rejection. Unauthorized actions and self-approval fail at contract level.
- Duplicate activity IDs and repeated decisions fail.
- Dashboard reflects actual contract state and transaction references.
- QR page verifies the actual on-chain record and fails safely when RPC is unavailable.
- Evidence and personal data are not public. Frontend build, backend tests and SmartPy tests run. Distinguish unit tests from live testnet tests.

10. Instructions to the coding agent

- Begin by inspecting the workspace and verifying current dependencies and testnet support. List files to create/change, then implement Phase 1 only.
- For every phase provide complete files, explain responsibilities, give exact PowerShell commands, run relevant checks, report actual results, fix errors and identify manual setup actions. Preserve working code.
- Do not leave required features as TODOs or fake transaction handlers. If wallet, RPC, faucet or deployment is unavailable, state what is missing and the next manual step.